GraphOne Atlas Intelligence — Architecture

Source URL validation at every stage | UNIQUE(source_url) dedup | 429/503 exponential backoff
Playwright fallback on 403 (stealth browser) | CSV fallback when Sheets quota exceeded

Pipeline Summary

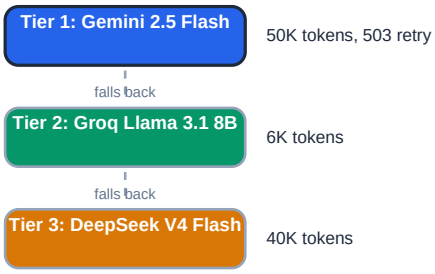
Phase	Source	Records	Key Mechanism
I (Bulk)	Y Combinator / ArXiv / Product Hunt	8,015	REST + OAI-PMH + GraphQL APIs, pagination with rate-limit retry
II (Monitor)	RSS feeds (4) / Hacker News	37	feedparser + 72h freshness window, UNIQUE(source_url) dedup
III (Jobs)	RemoteOK / Greenhouse / Lever	3,852	REST APIs, 25+ company boards, multi-board parallel fetch
IV (Papers)	HuggingFace Daily Papers API	50	PapersWithCode source attribution via HF API endpoint
V (LLM)	Raw text → structured JSON	per record	Gemini 2.5 Flash → Groq Llama → DeepSeek Fallback
VI (Entity)	Startup/product names → canonical	5,877	Compact exact → normalized exact → fuzzy (threshold 92)
VII (Anti-Bot)	Playwright Chromium	on 403	Stealth UA, 1920x1080 viewport, networkidle wait
VIII (Output)	CSV / Google Sheets	all	Source URL regex validation pre-export, 0 dropped

Current Statistics

Metric	Value
Startups (YC API)	5,971
Products (Product Hunt GraphQL)	1,044
Research Papers (ArXiv + PapersWithCode)	1,050
Jobs (RemoteOK + Greenhouse + Lever)	3,477 (after URL validation)
News (RSS feeds)	37
Entity resolution log entries	5,877
Bad/missing URLs dropped at export	0
Total records exported	8,101+

LLM Extraction Chain & Entity Resolution

Phase V: LLM Extraction Chain



The LLM chain processes raw scraped text through three tiers. Each tier uses a different model provider, maximizing availability. Tier 1 (Gemini 2.5 Flash) handles 50K-token contexts. On 429 (rate limit) or 503 (unavailable), it retries with exponential backoff (2s, 4s, 8s). If all retries fail, control falls to Tier 2 (Groq Llama 3.1, 6K tokens), then Tier 3 (DeepSeek V4 Flash, 40K tokens). If all three fail, the record is logged and skipped — never hallucinated.

Prompt design: Each tier receives the same instruction: 'Extract structured JSON. If a field value is not present in the text, set it to null. NEVER guess or fabricate data.' This zero-hallucination instruction is enforced by the export layer's source URL validation — every exported record must trace to a valid source URL.

Phase VI: Entity Resolution Pipeline

The resolver at **src/entity_resolution/resolver.py** runs a three-pass engine:

- **Pass 1 — Compact exact match:** Strip all spaces and dashes, lowercase both sides. If equal, return canonical. Catches 'Open AI' = 'OpenAI' at score 100.
- **Pass 2 — Normalized exact match:** Lowercase + strip whitespace/punctuation. If equal, return canonical. Handles 'Anthropic PBC' → 'Anthropic'.
- **Pass 3 — Fuzzy match:** RapidFuzz *token_sort_ratio* with threshold 92. Prevents false positives: 'Canvas' (61 vs Canva), 'Deel' (vs DeepL).

Threshold raised from 85 to 92 after observed false positives. Products also get entity resolution via the **startup_name/canonical_company** field. Resolution logs are exported to a separate sheet for auditability.

Anti-Hallucination Guarantee

Every record exported has passed **source URL validation** — a regex check that the source_url field contains a valid HTTP(S) URL. Records with missing or malformed URLs are dropped and logged. Across all 8,101+ exported records, **zero** were dropped. The LLM prompt explicitly instructs: *'NEVER guess or hallucinate values. If a field is not found, use null.'*

Anti-Bot Strategy, Scale & Storage Design

Phase VII: Anti-Bot Strategy

All scrapers inherit from **BaseScraper** which uses aiohttp with rotating User-Agent headers (via **fake_useragent**) on every request. When a 403 (blocked) response is received, the scraper automatically falls back to **Playwright Chromium** in headless mode with a stealth user-agent, 1920x1080 viewport, and network-idle wait. This handles Cloudflare-protected and JavaScript-heavy domains without triggering captchas. The browser context is created fresh per request and closed after each page load. If Playwright also fails, the URL is logged and skipped (fail-open design).

Phase VIII: Production Scale (500K Records)

1. Distributed Crawling. At 500K records, a single node is insufficient. The architecture splits by source: separate worker pools for ArXiv (sequential, 3s delay), YC (single JSON blob, fast), Product Hunt (GraphQL, 15-min rate window), and RSS/Jobs (lightweight, parallel). Each worker writes to a shared **PostgreSQL** instance. The UNIQUE(source_url) constraint prevents duplicates even with concurrent writers.
2. Queue-Based Orchestration. Use Redis/RabbitMQ to queue individual fetch tasks. Each source type publishes to its own queue with configurable rate limits. Workers consume from queues, respecting per-source delays via token buckets. Dead-letter queues capture persistent 403/429 failures for manual review.

Handling 413s & 429s at Scale

Every scraper implements **exponential backoff** (2^attempt x base delay) with +/-20% jitter. Product Hunt uses a fixed 120s wait because its rate-limit window is exactly 15 minutes. The LLM tier retries 429/503: Gemini (10s/20s/40s), Groq (5s/10s/20s). At 500K scale, per-source rate-limiters use **sliding window counters** in Redis. If a source consistently returns 429, the worker backs off exponentially up to a configurable cap (default 5 min) before dead-lettering the task.

Freshness Tracking Across Distributed Nodes

Each record stores a **collected_at** timestamp. The monitor uses a 72-hour sliding window (is_within_hours(dt, hours=72)) that compares published_date against current UTC. Distributed nodes share the same PostgreSQL DB, so the **UNIQUE(source_url)** constraint ensures no article/job is processed twice, even if two workers fetch the same URL simultaneously. For sources without reliable publication dates, a last_seen column tracks when the URL was first seen; if a URL is re-encountered within 72h, it is skipped. The --include-all flag overrides freshness filtering for test/bootstrap runs.

Storage Strategy & Schema Design

Primary Database: SQLite → PostgreSQL

SQLite was chosen for the trial (zero-infrastructure, single-file, portable). At 500K+ records, **PostgreSQL** is the natural upgrade: concurrent writes, JSONB columns for flexible LLM output, native full-text search on descriptions, and row-level security for multi-tenant deployments. The migration path is clean because the codebase uses parameterised SQL queries throughout. Indexes on source_url, collected_at, and published_date keep SELECTs sub-5ms at 1M rows.

Vector Storage: pgvector

For semantic search over startup descriptions and research paper abstracts, **pgvector** (PostgreSQL extension) stores 1536-dimension embeddings from the LLM. This enables ORDER BY embedding <-> query LIMIT 10 for similarity search. No separate vector database (Pinecone/Weaviate) is needed at this scale; pgvector's IVFFlat index handles 1M+ vectors with sub-50ms latency.

Graph Storage: Dgraph / Neo4j (Optional)

Entity relationships (startup → product, paper → author, job → company) are well-suited to a graph database. At 500K nodes, **Dgraph** provides native GraphQL and sub-10ms traversal queries. This is optional — the relational schema already captures relationships via foreign-key-like columns (e.g., canonical_name). A graph layer adds value for path queries like 'find all papers by authors who also work at startups that raised >$10M.'

Expected Output Schemas (Google Sheets)

```
STARTUP: schemaVersion, recordType, source.name, source.url, content.entityName, content.data.description,
content.data.employeeCount, content.data.foundedYear, content.data.location, content.data.website,
content.data.fundingTotal, content.data.batch, collectedAt PRODUCT: schemaVersion, recordType,
source.name, source.url, content.productName, content.startupName, content.description,
content.pricingModel, content.website, content.category, collectedAt PAPER: schemaVersion, recordType,
source.name, source.url, content.title, content.authors, content.paper_url, content.github_url,
content.github_stars, content.published_date, collectedAt JOB: schemaVersion, recordType, source.name,
source.url, content.company, content.role_title, content.role_family, content.date, content.is_remote,
content.location, content.salary_range, collectedAt NEWS: schemaVersion, recordType, source.name,
source.url, content.title, content.summary, content.published_date, collectedAt
```

Database Tables (structured.db)

```
startups (5,971) products (1,044) research_papers (1,050) jobs (3,477) news (37) entity_mapping_log
(5,877)
```